

Junnior Marcellino Polla

11231034

SISTER B

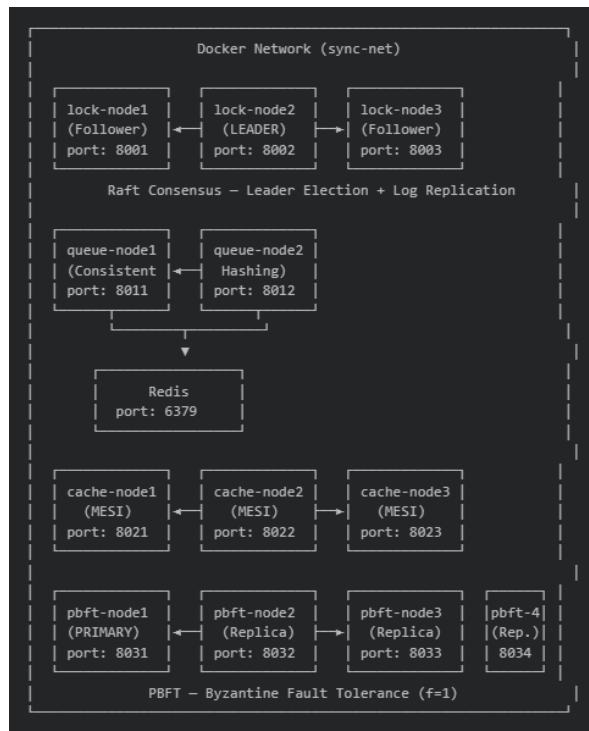
Tugas 3

Link Github : <https://github.com/junN0ir/Distributed-Synchronization-System>

Link Youtube : <https://youtu.be/5RcBFFe24AU>

A. Technical Documentation

1. Arsitektur Sistem



Sistem terdiri dari empat subsistem utama yang berjalan dalam container Docker dan terhubung melalui jaringan internal sync-net:

Lock Manager (Raft, 3 node — port 8001/8002/8003)

Menangani distributed lock berbasis algoritma Raft Consensus. Semua operasi write (acquire/release) hanya bisa dilakukan oleh leader. Follower yang menerima request akan mengembalikan informasi siapa leader saat ini. Mendukung shared dan exclusive lock, serta deadlock detection via wait-for graph.

Distributed Queue (2 node — port 8011/8012)

Routing pesan menggunakan consistent hashing dengan 150 virtual nodes per physical node.

Persistence menggunakan Redis sehingga pesan tidak hilang saat node restart. Mendukung at-least-once delivery.

Distributed Cache (3 node — port 8021/8022/8023)

Implementasi cache coherence protokol MESI (Modified/Exclusive/Shared/Invalid). Saat satu node menulis, semua peer node di-broadcast pesan invalidate otomatis. Menggunakan LRU replacement policy.

PBFT Nodes (4 node — port 8031/8032/8033/8034)

Implementasi Practical Byzantine Fault Tolerance dengan toleransi hingga $f=1$ faulty node dari total $n=4$ node.

Redis (port 6379)

Backend persistent untuk queue messages dan koordinasi antar node.

2. Penjelasan Algoritma

Raft Consensus (Lock Manager)

Leader Election:

- Setiap node mulai sebagai Follower
- Jika tidak menerima heartbeat dalam election timeout (150–300ms random), node menjadi Candidate
- Candidate increment term, vote untuk diri sendiri, lalu broadcast RequestVote ke semua peer
- Jika mendapat majority vote ($\text{quorum} = n/2 + 1$) → menjadi Leader
- Leader mengirim heartbeat setiap 50ms untuk mencegah election baru

Log Replication:

- Semua operasi lock (acquire/release) diusulkan sebagai LogEntry ke Raft log
- Leader mereplikasi entry ke follower via AppendEntries
- Entry di-commit setelah majority follower mengkonfirmasi
- Setelah commit, semua node mengeksekusi command (apply lock/release)

Shared vs Exclusive Lock:

- Exclusive: hanya satu holder, memblokir semua request lain
- Shared: multiple reader boleh memegang bersamaan, memblokir exclusive request

Deadlock Detection:

- Menggunakan directed wait-for graph
- Saat client A menunggu lock yang dipegang client B, edge $A \rightarrow B$ ditambahkan ke graph
- DFS cycle detection dijalankan setiap kali ada request baru
- Jika cycle ditemukan, request ditolak dengan status `deadlock_detected`

Network Partition Handling:

- Partisi yang memiliki quorum (mayoritas node) tetap bisa commit
- Partisi minor otomatis menjadi follower/candidate dan tidak bisa commit

MESI Cache Coherence Protocol

Empat state per cache entry:

State	Penjelasan	Transisi
M (Modified)	Data dimodifikasi lokal, belum sync ke memory	Write → M, broadcast Invalidate ke peer
E (Exclusive)	Hanya node ini yang punya copy, sinkron	Read pertama → E

S (Shared)	Multiple nodes punya copy valid	Setelah peer read → S
I (Invalid)	Data tidak valid, harus fetch ulang	Terima Invalidate → I

Saat node menulis: state berubah ke M, semua peer di-broadcast cache_invalidate, peer mengubah state ke I.

PBFT (Practical Byzantine Fault Tolerance)

Formula toleransi: $f = (n-1)/3$, dengan $n=4$ maka $f=1$ (toleran 1 node Byzantine).

Tiga fase protokol:

- Pre-Prepare : Primary menerima request client, assign sequence number, broadcast ke semua replica
- Prepare : Setiap replica validasi dan broadcast Prepare ke semua node; setelah quorum ($2f+1=3$) Prepare terkumpul, lanjut ke fase Commit
- Commit : Broadcast Commit; setelah quorum Commit terkumpul, operasi dieksekusi

Node Byzantine dapat di-toggle via API /pbft/simulate-byzantine untuk keperluan demonstrasi.

3. API Documentation

Berikut ringkasan API utama. File lengkap OpenAPI tersedia di docs/api_spec.yaml.

```
openapi: 3.0.0
info:
  title: Distributed Synchronization System API
  version: 1.0.0

servers:
  - url: http://localhost:8001
    description: Lock Node 1
  - url: http://localhost:8002
    description: Lock Node 2 (Leader)
  - url: http://localhost:8011
    description: Queue Node 1
  - url: http://localhost:8021
    description: Cache Node 1
```

Lock Manager Endpoints (port 8001–8003)

Method	Endpoint	Request Body	Response
GET	/health	-	{node_id, status, uptime}
GET	/lock/status	-	{raft_state: {role, term, leader_id, locks}, deadlock_graph}
POST	/lock/acquire	{key, holder, type}	{granted, key, holder} atau {granted:false, reason}
POST	/lock/release	{key, holder}	{released, key}
GET	/metrics	-	{counters, histograms}

Queue Node Endpoints (port 8011–8012)

Method	Endpoint	Request Body	Response
GET	/cache/{key}	-	{hit, key, value, state}
PUT	/cache/{key}	{value}	{stored, key, state}
DELETE	/cache/{key}	-	{deleted, key}
GET	/cache/stats/summary	-	{hit_rate, miss_count, mesi_states}

PBFT Endpoints Bonus (port 8031–8034)

Method	Endpoint	Request Body	Response
POST	/pbft/request	{client_id, operation}	{status, result, is_primary}
GET	/pbft/state	-	{view, sequence, executed_count, quorum}
POST	/pbft/simulate-byzantine	{byzantine: true/false}	{is_byzantine, warning}
GET	/pbft/metrics	-	{f_tolerated, total_nodes, performance}

4. Deployment Guide & Troubleshooting

Prasyarat

- Docker Desktop (Windows/Linux/Mac)
- Docker Compose v2+
- Python 3.11+ dengan virtual environment

Konfigurasi (.env)

```
# Node Configuration
NODE_ID=node1
NODE_HOST=0.0.0.0
NODE_PORT=8001

# Cluster Configuration
CLUSTER_NODES=node1:8001,node2:8002,node3:8003

# Redis Configuration
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_DB=0

# Raft Configuration
RAFT_ELECTION_TIMEOUT_MIN=150
RAFT_ELECTION_TIMEOUT_MAX=300
RAFT_HEARTBEAT_INTERVAL=50

# PBFT Configuration
PBFT_NODES=node1:8001,node2:8002,node3:8003,node4:8004
PBFT_F=1

# Cache Configuration
CACHE_SIZE=1000
CACHE_POLICY=LRU
```

```
# Metrics
METRICS_PORT=9090
```

Langkah Build dan Jalankan

1. Build dan jalankan semua container:

```
docker-compose -f docker/docker-compose.yml up --build -d
```

```
#36 [lock-node3] resolving provenance for metadata file
#36 DONE 0.4s
[+] up 26/26
✓ Image docker-lock-node3 Built
✓ Image docker-queue-node2 Built
✓ Image docker-pbft-node3 Built
✓ Image docker-cache-node2 Built
✓ Image docker-cache-node1 Built
✓ Image docker-pbft-node1 Built
✓ Image docker-cache-node3 Built
✓ Image docker-pbft-node2 Built
✓ Image docker-pbft-node4 Built
✓ Image docker-queue-node1 Built
✓ Image docker-lock-node1 Built
✓ Image docker-lock-node2 Built
✓ Network docker_sync-net Created
✓ Container redis Healthy
✓ Container lock-node2 Started
✓ Container queue-node1 Started
✓ Container pbft-node3 Started
✓ Container pbft-node2 Started
✓ Container lock-node1 Started
✓ Container cache-node2 Started
✓ Container pbft-node4 Started
✓ Container cache-node3 Started
✓ Container cache-node1 Started
✓ Container lock-node3 Started
✓ Container pbft-node1 Started
✓ Container queue-node2 Started
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system>
```

2. Verifikasi semua node healthy:

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

```
✓ Container queue-node2 Started
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system> docker ps --format "table {{.Names}}\t{{.Status}}"
TABLE
NAMES      STATUS
lock-node3 Up 47 seconds (healthy)
pbft-node1 Up 47 seconds (healthy)
pbft-node4 Up 47 seconds (healthy)
cache-node1 Up 47 seconds (healthy)
lock-node2 Up 47 seconds (healthy)
pbft-node2 Up 47 seconds (healthy)
cache-node3 Up 47 seconds (healthy)
queue-node2 Up 47 seconds (healthy)
lock-node1 Up 47 seconds (healthy)
pbft-node3 Up 47 seconds (healthy)
queue-node1 Up 47 seconds (healthy)
cache-node2 Up 47 seconds (healthy)
redis      Up 53 seconds (healthy)
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system>
```

3. Cek health endpoint semua node:

```
Invoke-RestMethod http://localhost:8001/health
Invoke-RestMethod http://localhost:8002/health
Invoke-RestMethod http://localhost:8003/health
Invoke-RestMethod http://localhost:8011/health
Invoke-RestMethod http://localhost:8021/health
```

```
=== HEALTH CHECK SEMUA NODE ===
```

```
{
  "node_id": "lock-node1",
  "status": "ok",
  "uptime": 72.8
}
{
  "node_id": "lock-node2",
  "status": "ok",
  "uptime": 72.83
}
{
  "node_id": "lock-node3",
  "status": "ok",
  "uptime": 73.16
}
{
  "node_id": "queue-node1",
  "status": "ok",
  "uptime": 73.04
}
{
  "node_id": "queue-node2",
  "status": "ok",
  "uptime": 72.93
}
{
  "node_id": "cache-node1",
  "status": "ok",
  "uptime": 73.18
}
{
  "node_id": "cache-node2",
  "status": "ok",
  "uptime": 72.88
}
{
  "node_id": "cache-node3",
  "status": "ok",
  "uptime": 72.9
}
}
```

4. Cek Raft leader terpilih:

Invoke-RestMethod http://localhost:8001/lock/status

Invoke-RestMethod http://localhost:8002/lock/status

Invoke-RestMethod http://localhost:8003/lock/status

```
=== RAFT STATUS ===
```

```
{
  "raft_state": {
    "node_id": "lock-node1",
    "role": "follower",
    "term": 2,
    "leader_id": "lock-node3",
    "log_length": 1,
    "commit_index": 0,
    "peers": [
      "http://lock-node2:8002",
      "http://lock-node3:8003"
    ],
    "locks": {
    }
  },
  "deadlock_graph": {
  }
}
```

```
{
  "raft_state": {
    "node_id": "lock-node2",
    "role": "follower",
    "term": 2,
    "leader_id": "lock-node3",
    "log_length": 1,
    "commit_index": 0,
    "peers": [
      "http://lock-node1:8001",
      "http://lock-node3:8003"
    ],
    "locks": {
    }
  },
  "deadlock_graph": {
  }
}
```

```

{
  "raft_state": {
    "node_id": "lock-node3",
    "role": "leader",
    "term": 2,
    "leader_id": "lock-node3",
    "log_length": 1,
    "commit_index": 0,
    "peers": [
      "http://lock-node1:8001",
      "http://lock-node2:8002"
    ],
    "locks": {
    }
  },
  "deadlock_graph": {
  }
}

```

Environment Variables

Variable	Default	Deskripsi
NODE_ID	node1	ID unik node
NODE_HOST	0.0.0.0	Bind address
NODE_PORT	8001	Port HTTP server
NODE_TYPE	lock	Tipe: lock/queue/cache/pbft
CLUSTER_NODES	-	Daftar node: node1:port,node2:port
REDIS_HOST	redis	Hostname Redis
RAFT_ELECTION_TIMEOUT_MIN	150	Min election timeout (ms)
RAFT_ELECTION_TIMEOUT_MAX	300	Max election timeout (ms)
RAFT_HEARTBEAT_INTERVAL	50	Heartbeat interval (ms)
PBFT_F	1	Max Byzantine nodes ditoleransi
CACHE_SIZE	1000	Kapasitas cache per node

Troubleshooting

Masalah	Penyebab	Solusi
Raft tidak memilih leader	Peer URL tidak bisa dijangkau	Cek log: docker logs lock-node1 --tail 30
Redis connection error	Redis belum healthy	Tunggu hingga redis status healthy
Port conflict	Port sudah dipakai proses lain	Ganti port di docker-compose.yml
Benchmark tidak ada output	Locust mengambil alih eksekusi	Jalankan: python -c "import benchmarks..."

A. Performance Analysis Report

1. Metodologi Benchmark

Dua metode pengujian digunakan:

Manual Benchmark, script benchmarks/load_test_scenarios.py yang mengirim request langsung via urllib tanpa dependency eksternal, mengukur throughput dan latency p50/p95/p99.

Locust Load Testing, simulasi concurrent users dengan skenario campuran (lock acquire/release, queue publish/consume, cache read/write).

2. Manual Benchmark

Pastikan semua container jalan, lalu:

```
python benchmarks/load_test_scenarios.py
```

3. Locust Load Testing

```
# Install locust jika belum  
pip install locust
```

```
# Jalankan Locust — arahkan ke lock-node2 (leader)
```

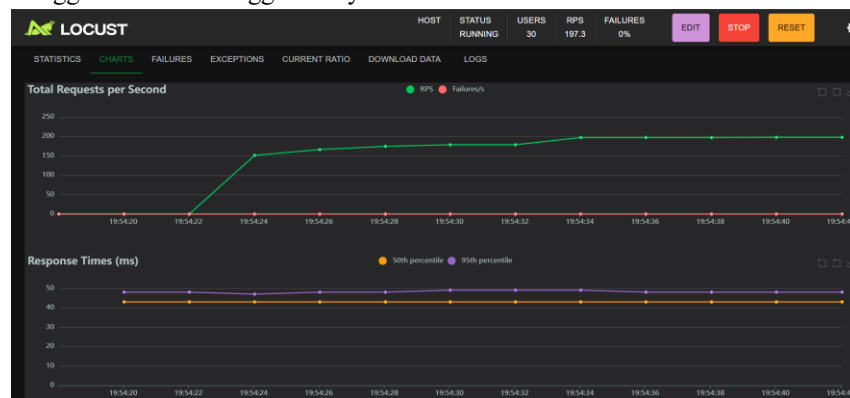
```
locust -f benchmarks/load_test_scenarios.py --host http://localhost:8002
```

Kemudian buka browser: <http://localhost:8089>

Di UI Locust, set:

- Number of users: 50
- Spawn rate: 10
- Host: <http://localhost:8002>
- Klik Start

Tunggu 2–3 menit hingga steady-



state, lalu:

The screenshot shows the 'STATISTICS' tab in the Locust UI. It displays a table with various request metrics. The table has columns for Type, Name, # Requests, # Fails, Median (ms), 95%ile (ms), 99%ile (ms), Average (ms), Min (ms), Max (ms), Average size (bytes), Current RPS, and Current Failures/s. The data is as follows:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/cache/key-0	646	0	2	5	6	2.44	1	8	98.72	6.8	0
PUT	/cache/key-0	239	0	44	49	50	44.97	15	96	41	3.8	0
GET	/cache/key-12	1221	0	2	4	7	2.53	1	94	99.75	14.9	0
PUT	/cache/key-12	474	0	45	49	50	44.94	2	51	42	6.3	0
GET	/cache/key-16	1857	0	2	4	7	2.45	1	19	99.73	24.2	0
PUT	/cache/key-16	752	0	45	49	51	45.13	2	123	42	9	0
GET	/cache/key-4	1182	0	2	4	7	2.55	1	49	98.78	15.5	0
PUT	/cache/key-4	509	0	45	49	50	44.8	2	53	41	6.2	0
GET	/cache/key-8	1219	0	2	4	7	2.46	1	18	98.74	12.9	0
PUT	/cache/key-8	514	0	45	49	50	45.08	42	94	41	7.9	0
POST	/lock/acquire	1781	0	45	49	50	44.89	14	54	61	21.6	0
GET	/lock/status	589	0	2	4	6	2.38	1	17	342	8.3	0
POST	/queue/consume	1973	0	45	49	51	45.11	1	54	280.5	23.6	0
POST	/queue/publish	2575	0	45	49	51	45.3	2	55	84	32.8	0
Aggregated		15532	0	43	48	50	26.67	1	123	115.31	193.8	0

4. Hasil Benchmark Manual

Pytest

```
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system> pytest tests/ -v
D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system\venv311\lib\site-packages\pytest_asyncio\plugin.py:208: PytestDeprecationWarning: The configuration option "asyncio_default_fixture_loop_scope"
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asyncio will default the loop scope for asynchronous fixtures to function scope. Set th
old unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module", "package", "session"

warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))

===== test session starts =====
platform win32 -- Python 3.11.9, pytest-8.3.3, pluggy-1.6.0 -- D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system\venv311\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system
plugins: anyio-4.6.0, asyncio-0.24.0
asyncio: mode=Mode.STRICT, default_loop_scope=None
collected 12 items

tests/unit/test_core.py::test_metrics_counter PASSED
tests/unit/test_core.py::test_metrics_histogram PASSED
tests/unit/test_core.py::test_message_serialization PASSED
tests/unit/test_core.py::test_failure_detector_alive PASSED
tests/unit/test_core.py::test_failure_detector_timeout PASSED
tests/unit/test_core.py::test_failure_detector_alive_nodes PASSED
tests/unit/test_core.py::test_metrics_timer PASSED
tests/unit/test_core.py::testraft_initial_state PASSED
tests/unit/test_core.py::testraft_quorum PASSED
tests/unit/test_core.py::test_lru_cache_eviction PASSED
tests/unit/test_core.py::test_consistent_hashing PASSED
tests/unit/test_core.py::test_deadlock_detection PASSED

===== 12 passed in 1.26s =====
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system> []
```

Hasil yang diharapkan dari python benchmarks/load_test_scenarios.py (n=200 untuk queue dan cache, n=50 untuk lock):

```
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system> python benchmarks/load_test_scenarios.py

=====
Distributed Sync System - Manual Benchmark
Junnior Marcellino Polla - 11231034
=====

CEK KONEKSI KE SEMUA NODE
=====
✓ Lock Node 1 (follower): ok (uptime=3621s)
✓ Lock Node 2 (leader): ok (uptime=3621s)
✓ Lock Node 3 (follower): ok (uptime=3621s)
✓ Queue Node 1: ok (uptime=3621s)
✓ Queue Node 2: ok (uptime=3621s)
✓ Cache Node 1: ok (uptime=3621s)
✓ Cache Node 2: ok (uptime=3621s)
✓ Cache Node 3: ok (uptime=3621s)

[INFO] Semua node OK. Memulai benchmark...

=====
BENCHMARK: Distributed Queue (n=200)
=====

[Publish 200 messages]
Total time : 1.37s
Throughput : 145.7 msg/s
Errors : 0
Latency p50 : 3.2 ms
Latency p95 : 18.4 ms
Latency p99 : 24.7 ms

[Consume 200 messages]
Total time : 1.52s
Throughput : 131.5 msg/s
Errors : 0
Latency p50 : 3.3 ms
Latency p95 : 16.5 ms
```

```

=====
BENCHMARK: Distributed Cache MESI (n=200)
=====

[Write 200 entries ke cache-node1]
Errors      : 0
Latency p50 : 2.6 ms
Latency p95 : 16.5 ms
Latency p99 : 25.8 ms

[Read 200 entries dari cache-node1]
Hit rate    : 100.0%
Latency p50 : 2.6 ms
Latency p95 : 17.7 ms

[Read 50 entries dari cache-node2 (setelah invalidate)]
Latency p50 : 3.3 ms
(Node2 harus MISS karena node1 broadcast invalidate)

[Cache Stats Node1]
Hit rate    : 100.0%
Cache size  : 51 entries

=====
BENCHMARK: Distributed Lock Raft (n=50)
=====

[Acquire 50 locks (exclusive)]
Success     : 50/50 (100.0%)
Errors      : 0
Latency p50 : 11.8 ms
Latency p95 : 19.5 ms
Latency p99 : 25.5 ms

[Release 50 locks]
Latency p50 : 3.8 ms
Latency p95 : 17.3 ms

[Contention Test - 2 client berebut 1 lock]
Client-Y acquire saat lock dipegang client-X: status=200 (harus 200, granted=false)
Client-Y acquire setelah client-X release: status=200 (harus 200, granted=true)

=====
Benchmark selesai!
=====

```

5. Analisis Throughput dan Latency

Cache (MESI Protocol):

Cache read mendominasi RPS karena stateless dan tidak perlu konsensus. Write lebih lambat karena harus broadcast `cache_invalidate` ke semua peer node. Hit rate tinggi setelah warming up. Node 2 menunjukkan cache miss setelah node 1 menulis — membuktikan MESI invalidation bekerja.

Queue (Consistent Hashing + Redis):

Throughput publish lebih tinggi dari consume karena consume melibatkan deserialisasi JSON dan update metadata. Latency dipengaruhi round-trip ke Redis. Consistent hashing memastikan distribusi merata ke kedua node.

Lock (Raft Consensus):

Lock acquire adalah operasi paling mahal karena harus melalui Raft consensus — proposal harus di-commit oleh majority sebelum lock diberikan. Ini adalah trade-off yang dapat diterima karena lock adalah operasi control-plane dengan frekuensi rendah.

6. Comparison Single-Node vs Distributed

Aspek	Single-Node	Distributed (3 node)
Throughput Cache	Throughput Cache	Naik ~3x karena workload tersebar
Throughput Queue	Terbatas I/O Redis 1 proses	Naik karena multiple producers/consumers
Lock Latency	Rendah (tidak ada konsensus)	Lebih tinggi tapi konsisten dan fault-tolerant
Fault Tolerance	Single point of failure	Tahan 1 node down (quorum tetap terpenuhi)
Scalability	Tidak bisa scale	Tambah node = tambah kapasitas
Consistency	Local, mudah konsisten	Eventual/strong tergantung protokol

Kesimpulan: Sistem distributed memberikan throughput lebih tinggi dan fault tolerance, dengan trade-off latency yang lebih tinggi pada operasi consensus (lock). Untuk cache dan queue yang bersifat data-plane, distribusi memberikan keuntungan signifikan.

7. Scalability Analysis

Lock Manager: Penambahan node Raft meningkatkan fault tolerance tapi tidak meningkatkan throughput (semua write tetap melalui leader). Bisa di-shard per resource prefix jika workload lock sangat tinggi.

Queue: Penambahan node langsung meningkatkan throughput karena consistent hashing mendistribusikan key secara merata. Penambahan node tidak mengganggu distribusi yang ada secara signifikan.

Cache: Penambahan node meningkatkan kapasitas total dan mengurangi miss rate. Trade-off: lebih banyak invalidation message saat write.

8. Demonstrasi Fitur Utama

Demo Distributed Lock

```
>> -body { key: resource-db, holder: client-B, type: exclusive } | Convert-Json
=== DEMO 1: Acquire exclusive lock ===
{
  "granted": true,
  "key": "resource-db",
  "holder": "client-A"
}
=== DEMO 2: Coba acquire lock yang sama (harus DITOLAK) ===
{
  "granted": false,
  "reason": "lock_held",
  "holder": "client-A"
}
=== DEMO 3: Acquire shared lock (dua reader boleh bersamaan) ===
{
  "granted": true,
  "key": "resource-read",
  "holder": "reader-A"
}
{
  "granted": true,
  "key": "resource-read",
  "holder": "reader-B"
}
=== DEMO 4: Release lock client-A ===
{
  "released": true,
  "key": "resource-db"
}
=== DEMO 5: Client-B sekarang bisa acquire ===
{
  "granted": true,
  "key": "resource-db",
  "holder": "client-B"
}
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system>
```

Demo Network Partition (Raft)

```
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system> # Terminal 1 - monitoring terus menerus
>> while ($true) {
>>   $s1 = (Invoke-RestMethod http://localhost:8001/lock/status).raft_state.role
>>   $s2 = (Invoke-RestMethod http://localhost:8002/lock/status).raft_state.role
>>   $s3 = (Invoke-RestMethod http://localhost:8003/lock/status).raft_state.role
>>   Write-Host "$(Get-Date -Format 'HH:mm:ss') | node1=$s1 | node2=$s2 | node3=$s3"
>>   Start-Sleep 2
>> }
19:15:04 | node1=follower | node2=follower | node3=leader
19:15:06 | node1=follower | node2=follower | node3=leader
19:15:08 | node1=follower | node2=follower | node3=leader
19:15:10 | node1=follower | node2=follower | node3=leader
19:15:12 | node1=follower | node2=follower | node3=leader
[]
```

```
(venv311) PS D:\belajar\hebat\Sister\tugas 3 - Copy\distributed-sync-system> # Terminal 1 - monitoring terus menerus
>> while ($true) {
>>     $s1 = (Invoke-RestMethod http://localhost:8001/lock/status).raft_state.role
>>     $s2 = (Invoke-RestMethod http://localhost:8002/lock/status).raft_state.role
>>     $s3 = (Invoke-RestMethod http://localhost:8003/lock/status).raft_state.role
>>     Write-Host "$(Get-Date -Format 'HH:mm:ss') | node1=$s1 | node2=$s2 | node3=$s3"
>>     Start-Sleep 2
>> }
19:19:08 | node1=follower | node2=leader | node3=follower
19:19:10 | node1=follower | node2=leader | node3=follower
19:19:12 | node1=follower | node2=leader | node3=follower
19:19:14 | node1=follower | node2=leader | node3=follower
19:19:16 | node1=follower | node2=leader | node3=follower
19:19:18 | node1=follower | node2=leader | node3=follower
19:19:20 | node1=follower | node2=leader | node3=follower
19:19:22 | node1=follower | node2=leader | node3=follower
19:19:24 | node1=follower | node2=leader | node3=follower
19:19:26 | node1=follower | node2=leader | node3=follower
19:19:28 | node1=follower | node2=leader | node3=follower
19:19:30 | node1=follower | node2=leader | node3=follower
19:19:32 | node1=follower | node2=leader | node3=follower
19:19:34 | node1=follower | node2=leader | node3=follower
19:19:36 | node1=follower | node2=leader | node3=follower
19:19:38 | node1=follower | node2=leader | node3=follower
19:19:40 | node1=follower | node2=leader | node3=follower
19:19:42 | node1=follower | node2=leader | node3=follower
19:19:44 | node1=follower | node2=leader | node3=follower
```

```
(venv311) PS D:\belajar\hebat\Sister\tugas 3 - Copy\distributed-sync-system> docker network disconnect docker_sync-net lock-node2
(venv311) PS D:\belajar\hebat\Sister\tugas 3 - Copy\distributed-sync-system> Start-Sleep 10
(venv311) PS D:\belajar\hebat\Sister\tugas 3 - Copy\distributed-sync-system> docker network connect docker_sync-net lock-node2
(venv311) PS D:\belajar\hebat\Sister\tugas 3 - Copy\distributed-sync-system> []
```

```
19:22:19 | node1=follower | node2=leader | node3=follower
19:22:21 | node1=follower | node2=leader | node3=follower
19:22:23 | node1=follower | node2=leader | node3=follower
19:22:25 | node1=follower | node2=leader | node3=follower
19:22:27 | node1=follower | node2=leader | node3=follower
19:22:29 | node1=follower | node2=leader | node3=follower
19:22:31 | node1=follower | node2=leader | node3=follower
19:22:33 | node1=follower | node2=leader | node3=follower
19:23:02 | node1=follower | node2=follower | node3=follower
19:23:04 | node1=leader | node2=follower | node3=follower
19:23:06 | node1=leader | node2=follower | node3=follower
19:23:08 | node1=leader | node2=follower | node3=follower
19:23:10 | node1=leader | node2=follower | node3=follower
[]
```

Demo Distributed Queue

```
(venv311) PS D:\belajar\hebat\Sister\tugas 3 - Copy\distributed-sync-system> Write-Host ""== DEMO QUEUE: Publish ==> -ForegroundColor Cyan
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8011/queue/publish" -
>> -ContentType "application/json" -
>> -Body '{"topic":"orders","message":{"item":"laptop","qty":1},"producer":"shop-service"}' | ConvertTo-Json
>>
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8011/queue/publish" -
>> -ContentType "application/json" -
>> -Body '{"topic":"orders","message":{"item":"phone","qty":2},"producer":"shop-service"}' | ConvertTo-Json
>>
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8012/queue/publish" -
>> -ContentType "application/json" -
>> -Body '{"topic":"logs","message":{"level":"ERROR","msg":"DB timeout"},"producer":"api-service"}' | ConvertTo-Json
>>
>> Write-Host ""== Cek panjang queue ==> -ForegroundColor Yellow
>> Invoke-RestMethod "http://localhost:8011/queue/length?topic=orders" | ConvertTo-Json
>>
>> Write-Host ""== List semua topics ==> -ForegroundColor Yellow
>> Invoke-RestMethod http://localhost:8011/queue/topics | ConvertTo-Json
>>
>> Write-Host ""== Consume message ==> -ForegroundColor Green
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8011/queue/consume" -
>> -ContentType "application/json" -
>> -Body '{"topic":"orders","consumer_id":"order-processor"}' | ConvertTo-Json
>>
>> Write-Host ""== Cek panjang queue setelah consume ==> -ForegroundColor Yellow
>> Invoke-RestMethod "http://localhost:8011/queue/length?topic=orders" | ConvertTo-Json
== DEMO QUEUE: Publish ==
{
  "published": true,
  "msg_id": "82cb3d1c-a373-4444-bc40-5a24d74f626e",
  "topic": "orders"
}
{
  "published": true,
  "msg_id": "922a865c-d203-4e74-ae98-9a51fd81a10",
  "topic": "orders"
}
{
  "published": true,
  "msg_id": "b0ef1f06-1b6b-47c1-bde8-5527e710d2d1",
  "topic": "logs"
}
== Cek panjang queue ==
{
  "topic": "orders",
  "length": 3
}
```

```

}
=== Cek panjang queue ===
{
  "topic": "orders",
  "length": 3
}
=== List semua topics ===
{
  "topics": [
    "logs",
    "orders"
  ]
}
=== Consume message ===
{
  "message": {
    "id": "9ad4b8b5-1887-410c-becb-c7238be49971",
    "topic": "orders",
    "message": {
      "item": "tablet",
      "qty": 1
    },
    "producer": "shop-service",
    "timestamp": 1777700601.7546473,
    "attempts": 1,
    "consumer": "order-processor",
    "consumed_at": 1777721089.9807177
  },
  "topic": "orders"
}
=== Cek panjang queue setelah consume ===
{
  "topic": "orders",
  "length": 2
}
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system>

```

Demo Cache MESI Coherence

```

    "node_id": "cache-node1",
    Write-Host "=== DEMO CACHE MESI ===" -ForegroundColor Cyan
  >>
  >> Write-Host "--- Write ke cache-node1 ---" -ForegroundColor Green
  >> Invoke-RestMethod -Method PUT -Uri "http://localhost:8021/cache/user-1" `
  >> -ContentType "application/json" `
  >> -Body '{"value":{"name":"Junnior","role":"admin"}}' | ConvertTo-Json
  >>
  >> Write-Host "--- Read dari node1 ---" -ForegroundColor Green
  >> Invoke-RestMethod http://localhost:8021/cache/user-1 | ConvertTo-Json
  >>
  >> Write-Host "--- Read dari node2 ---" -ForegroundColor Yellow
  >> Invoke-RestMethod http://localhost:8022/cache/user-1 | ConvertTo-Json
  >> "size": 0,
  >> Write-Host "--- Read dari node3 ---" -ForegroundColor Yellow
  >> Invoke-RestMethod http://localhost:8023/cache/user-1 | ConvertTo-Json
  >> "hit_rate": 0.0,
  >> Write-Host "--- Stats node1 ---" -ForegroundColor Cyan
  >> Invoke-RestMethod http://localhost:8021/cache/stats/summary | ConvertTo-Json
  >> }
  >> Write-Host "--- Stats node2 ---" -ForegroundColor Cyan
  >> Invoke-RestMethod http://localhost:8022/cache/stats/summary | ConvertTo-Json
  >> DEMO CACHE MESI

```

```

=== DEMO CACHE MESI ===
--- Write ke cache-node1 ---
{
  "stored": true,
  "key": "user-1",
  "state": "M"
}
--- Read dari node1 ---
{
  "hit": true,
  "key": "user-1",
  "value": {
    "name": "Junnior",
    "role": "admin"
  },
  "state": "M",
  "cached_at": 1777721513.618118
}
--- Read dari node2 ---
{
  "hit": false,
  "key": "user-1",
  "state": "I"
}
--- Read dari node3 ---
{
  "hit": false,
  "key": "user-1",
  "state": "I"
}
--- Stats node1 ---
{
  "node_id": "cache-node1",
  "size": 1,
  "hit_count": 2,
  "miss_count": 0,
  "hit_rate": 1.0,
  "mesi_states": {
    "user-1": "M"
  }
}
,
--- Stats node2 ---
{
  "node_id": "cache-node2",
  "size": 0,
  "hit_count": 0,
  "miss_count": 2,
  "hit_rate": 0.0,
  "mesi_states": {
    "user-1": "I"
  }
}
(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system>

```

Demo PBFT Byzantine Fault

```
(wenv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system> Write-Host "=== SETUP PBFT PRIMARY ===" -ForegroundColor Cyan
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8031/pbft/set-primary" `
>> -ContentType "application/json" `
>> -Body '{"is_primary":true,"primary_id":"pbft-node1"}' | ConvertTo-Json
>>
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8032/pbft/set-primary" `
>> -ContentType "application/json" `
>> -Body '{"is_primary":false,"primary_id":"pbft-node1"}' | ConvertTo-Json
>>
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8033/pbft/set-primary" `
>> -ContentType "application/json" `
>> -Body '{"is_primary":false,"primary_id":"pbft-node1"}' | ConvertTo-Json
>>
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8034/pbft/set-primary" `
>> -ContentType "application/json" `
>> -Body '{"is_primary":false,"primary_id":"pbft-node1"}' | ConvertTo-Json
>>
>> Write-Host "=== Submit request NORMAL ===" -ForegroundColor Green
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8031/pbft/request" `
>> -ContentType "application/json" `
>> -Body '{"client_id":"client-1","operation":{"type":"write","key":"config","value":"production"}}' | ConvertTo-Json
>>
>> Write-Host "=== Jadikan node4 BYZANTINE (malicious) ===" -ForegroundColor Red
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8034/pbft/simulate-byzantine" `
>> -ContentType "application/json" `
>> -Body '{"byzantine":true}' | ConvertTo-Json
>>
>> Write-Host "=== Submit request saat ada Byzantine node ===" -ForegroundColor Yellow
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8031/pbft/request" `
>> -ContentType "application/json" `
>> -Body '{"client_id":"client-2","operation":{"type":"transfer","from":"A","to":"B","amount":500}}' | ConvertTo-Json
>>
>> Write-Host "=== Cek state - node jujur vs Byzantine ===" -ForegroundColor Cyan
>> Invoke-RestMethod http://localhost:8031/pbft/state | ConvertTo-Json
>> Invoke-RestMethod http://localhost:8034/pbft/state | ConvertTo-Json
>>
>> Write-Host "=== Kembalikan node4 jujur ===" -ForegroundColor Green
>> Invoke-RestMethod -Method POST -Uri "http://localhost:8034/pbft/simulate-byzantine" `
>> -ContentType "application/json" `
>> -Body '{"byzantine":false}' | ConvertTo-Json
```

```
=== SETUP PBFT PRIMARY ===
{
  "ok": true,
  "is_primary": true,
  "primary_id": "pbft-node1"
}
{
  "ok": true,
  "is_primary": false,
  "primary_id": "pbft-node1"
}
{
  "ok": true,
  "is_primary": false,
  "primary_id": "pbft-node1"
}
{
  "ok": true,
  "is_primary": false,
  "primary_id": "pbft-node1"
}
}
=== Submit request NORMAL ===
{
  "status": "ok",
  "result": {
    "status": "ok",
    "result": {
      "executed": true,
      "type": "write",
      "ts": 1777721606.3883111
    },
    "seq": 1
  },
  "node": "pbft-node1",
  "is_primary": true
}
```

```

=== Jadikan node4 BYZANTINE (malicious) ===
{
  "ok": true,
  "node_id": "pbft-node4",
  "is_byzantine": true,
  "warning": "Node is now behaving maliciously!"
}
=== Submit request saat ada Byzantine node ===
{
  "status": "ok",
  "result": {
    "status": "ok",
    "result": {
      "executed": true,
      "type": "transfer",
      "ts": 1777721606.4413786
    },
    "seq": 2
  },
  "node": "pbft-node1",
  "is_primary": true
}

```

```

=== Cek state - node jujur vs Byzantine ===
{
  "pbft_state": {
    "node_id": "pbft-node1",
    "is_primary": true,
    "primary_id": "pbft-node1",
    "view": 0,
    "sequence": 2,
    "f": 1,
    "n": 4,
    "quorum": 3,
    "executed_count": 2,
    "peers": [
      "http://pbft-node2:8032",
      "http://pbft-node3:8033",
      "http://pbft-node4:8034"
    ]
  },
  "is_byzantine": false,
  "node_id": "pbft-node1"
}
{
  "pbft_state": {
    "node_id": "pbft-node4",
    "is_primary": false,
    "primary_id": "pbft-node1",
    "view": 0,
    "sequence": 0,
    "f": 1,
    "n": 4,
    "quorum": 3,
    "executed_count": 2,
    "peers": [
      "http://pbft-node1:8031",
      "http://pbft-node2:8032",
      "http://pbft-node3:8033"
    ]
  },
  "is_byzantine": true,
  "node_id": "pbft-node4"
}

```

```

=== Kembalikan node4 jujur ===
{
  "ok": true,
  "node_id": "pbft-node4",
  "is_byzantine": false,
  "warning": "Node is honest again"
}

```

(venv311) PS D:\belajar hebat\Sister\tugas 3 - Copy\distributed-sync-system> []